Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЕТ
по ЛАБОРАТОРНОЙ РАБОТЕ №3
на тему
«РАЗРАБОТКА И ДОКУМЕНТИРОВАНИЕ БИЗНЕС-ТРЕБОВАНИЙ»

Студент: Д.В. Мистюкович

Преподаватель: Д.В. Куприянова

Минск 2026

1 ЦЕЛИ И ЗАДАЧИ

Цель работы – изучить процесс выявления требований к программному обеспечению, освоить документирование требований с использованием SRS и получить практические навыки моделирования системы на языке SysML v2.

Для достижения поставленной цели необходимо решить следующие задачи:

1 Выбрать тему проекта и определить его назначение, границы и целевых пользователей.

2 Получить исходные требования заказчика, выявить неоднозначности и зафиксировать допущения.

3 Разработать спецификацию требований к веб-приложению «Retro SFX Lab» – генератору звуковых эффектов для игр с созданием 8-bit и ретро-звуков (прыжки, монеты, взрывы) и настройкой параметров.

4 Подготовить графические модели SysML v2: контекст системы, варианты использования, архитектуру приложения, состояния редактора звука, макеты интерфейса и цепочку синтеза звука.

5 Связать требования с критериями приёмки и способами верификации.

6 Оформить результаты работы и ответить на контрольные вопросы.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Требование к программному обеспечению и его виды

Требование к программному обеспечению – это условие, характеристика или возможность, которой должна обладать система для удовлетворения потребностей пользователя или достижения бизнес-цели. Требования могут описывать бизнес-цели, ожидаемое поведение системы, ограничения реализации и показатели качества.

Функциональные требования определяют, какие операции должна выполнять система. Нефункциональные требования задают характеристики выполнения этих операций: например, производительность, безопасность, надёжность, совместимость и сопровождаемость. Требование должно быть однозначным, проверяемым и согласованным с остальными требованиями.

2.2 Выявление и документирование требований

Выявление требований – итеративный процесс уточнения ожиданий заказчика и преобразования исходных пожеланий в согласованный набор требований. На начальном этапе часть условий может быть неизвестна или сформулирована неоднозначно, поэтому необходимо фиксировать допущения и открытые вопросы.

Для документирования используется спецификация требований к ПО (Software Requirements Specification, SRS). В документе описываются назначение продукта, границы, пользователи, ограничения, внешние интерфейсы, функциональные и нефункциональные требования, а также способы проверки выполнения требований. Уникальные идентификаторы позволяют связывать требования с тестами и другими артефактами проекта.

Критерии приёмки делают требование проверяемым: они определяют, по каким наблюдаемым результатам можно установить, выполнено ли требование. Для этого применяются тестирование, анализ, инспекция и демонстрация. Открытые вопросы должны быть выделены отдельно, чтобы предварительные решения не воспринимались как окончательно согласованные.

2.3 Прототипирование и язык SysML v2

Прототипы и графические модели позволяют согласовать ожидаемое поведение и структуру системы до начала полной реализации. SysML v2 используется для описания системного контекста, компонентов, вариантов использования и состояний поведения.

В данной работе диаграммы дополняют текст SRS: контекстная модель показывает участников и внешние взаимодействия, диаграмма вариантов использования отражает цели разработчика игры и дизайнера звука, архитектурная модель показывает независимое ядро синтеза, а диаграмма состояний описывает работу редактора. Отдельные макеты иллюстрируют интерфейс, а модель цепочки синтеза показывает преобразование параметров в звук.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Выбор темы проекта

В качестве темы выбрана разработка генератора звуковых эффектов «Retro SFX Lab» – веб-приложения для создания звуков в стиле игровых приставок 1980-х. Тип проекта – Web. Язык разработки ядра и бэкенда – C# (.NET 8), интерфейс реализуется на Blazor WebAssembly, серверная часть представлена ASP.NET Core API. Язык описания моделей и макетов – SysML v2.

Назначение приложения – быстро дать разработчику инди-игры нужный звук. Пользователь выбирает пресет (прыжок, монета, взрыв, лазер и другие) или собирает звук с нуля, настраивает параметры ползунками, сразу слышит результат, видит форму волны и экспортирует звук в файл WAV. Использование C# позволяет использовать созданное ядро синтеза напрямую в игровых движках Unity и Godot. Одинаковые параметры и seed всегда дают одинаковый файл, поэтому удачные звуки можно сохранять в JSON и воспроизводить повторно.

В область работы входят синтез по параметрам (форма волны, скважность, частота, огибающая ADSR, скольжение, вибрато, арпеджио, фильтры, bitcrush), пресеты, случайная генерация и мутация, воспроизведение, визуализация формы волны, история изменений, экспорт в WAV, сохранение и загрузка параметров. Запись с микрофона, обработка чужих аудиофайлов, секвенсор, экспорт в MP3 и OGG, учётные записи и мобильные приложения не входят в текущую область.

3.2 Исходные требования заказчика

Исходные пожелания заказчика, полученные в ходе ролевой игры с напарником, были преобразованы в набор бизнес-целей и системных требований. Основные ожидания сгруппированы следующим образом:

– пользователь выбирает готовый пресет (прыжок, монета, взрыв, лазер и др.) и сразу слышит звук;

– звук настраивается параметрами: форма волны, скважность, частота, огибающая ADSR, скольжение, вибрато, арпеджио;

– доступны фильтры, bitcrush и ограничение громкости, чтобы звук получался «ретро» и безопасным для слуха;

– кнопки случайной генерации и мутации создают вариации звука;

– форма волны отображается на графике и обновляется при изменении параметров;

– звук экспортируется в WAV для игрового движка, параметры сохраняются в JSON;

– одинаковые параметры дают одинаковый результат.

В ходе детализации были зафиксированы ограничения: приложение работает в браузере (Chrome, Firefox, Edge), параметры и звуки не покидают компьютер пользователя (stateless API), платные сервисы не используются. Длительность звука ограничена 5 секундами, формат экспорта – WAV PCM моно 16 бит. Файл параметров – внешний ввод, поэтому проверяются его размер, типы и диапазоны значений.

Отдельно определены сценарии загрузки некорректного файла параметров, значений вне допустимых диапазонов, блокировки воспроизведения политикой браузера, отсутствия Web Audio API (воспроизведение отключается, но экспорт работает), слишком громких звуков и повторного нажатия «Play» во время воспроизведения. Вопросы, требующие подтверждения заказчика, вынесены в приложение SRS.

3.3 Документ спецификации требований

Спецификация подготовлена в файле `srs.md` на основе предоставленного шаблона. В ней описаны назначение и границы системы, пользователи, допущения, внешние интерфейсы, требования к поведению и качеству, ограничения реализации, проверка требований и открытые вопросы.

Каждое системное требование имеет уникальный идентификатор, формулировку, обоснование, критерии приёмки и метод верификации. В текущей редакции зафиксировано 51 требование. Их распределение по областям приведено в таблице 3.1.

Таблица 3.1 – Распределение требований по областям

Область	Идентификаторы	Кол-во
Интерфейсы и интеграция	REQ-INT-001 – REQ-INT-008	8
Функциональные	REQ-FUNC-001 – REQ-FUNC-023	23
Производительность	REQ-PERF-001 – REQ-PERF-003	3
Безопасность	REQ-SEC-001 – REQ-SEC-002	2
Надёжность	REQ-REL-001 – REQ-REL-003	3
Наблюдаемость	REQ-OBS-001	1
Соответствие стандартам	REQ-COMP-001 – REQ-COMP-003	3
Сопровождаемость	REQ-MAINT-001 – REQ-MAINT-002	2
Повторное использование	REQ-REUSE-001 – REQ-REUSE-004	4
Переносимость	REQ-PORT-001	1
Доказательство концепции	REQ-POC-001	1
Итого	—	51

Проверяемость требований обеспечивается конкретными критериями приёмки. Например, синтез одной секунды звука на C# должен занимать не бо-

лее 100 мс, обновление графика после изменения параметра – не более 100 мс, а экспорт пятисекундного звука – не более 500 мс. Воспроизводимость проверяется сравнением контрольных сумм SHA-256 двух экспортированных файлов, корректность WAV – открытием в сторонних программах. Для надёжности предусмотрены негативные тесты с повреждёнными файлами параметров.

3.4 Графическая часть: модели SysML v2

Графическая часть состоит из моделей контекста, вариантов использования, архитектуры, состояний, пользовательского интерфейса и цепочки синтеза звука. Исходные текстовые модели сохранены в формате `.sysml`, а их графическое представление – в PNG.

3.4.1 Контекст системы

Контекстная диаграмма показывает разработчика игры и дизайнера звука, которые работают с редактором, и компоненты системы: интерфейс, ядро синтеза, проигрыватель и кодировщик WAV. Отображены внешние взаимодействия: вывод звука на устройство, сохранение WAV и JSON в файловой системе и последующий импорт WAV в игровой движок.

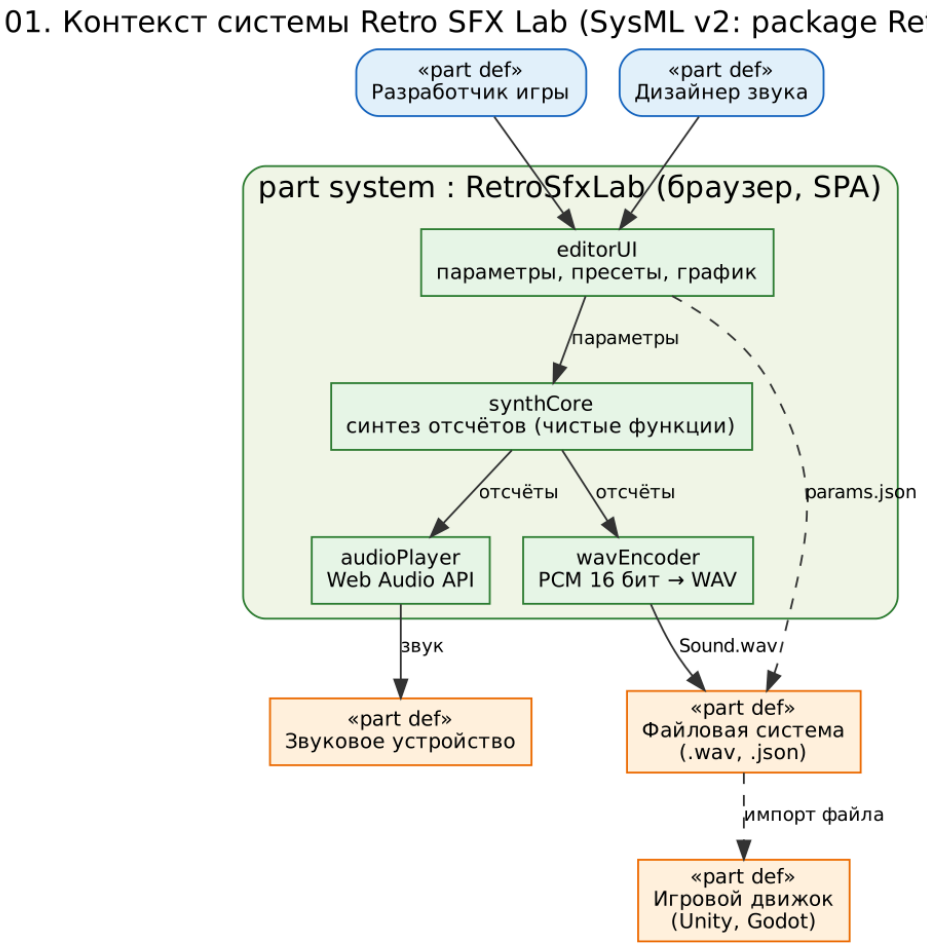


Рисунок 3.1 – Контекст системы Retro SFX Lab на языке SysML v2

3.4.2 Варианты использования

Диаграмма вариантов использования отражает сценарии пользователя: выбрать пресет, настроить параметры, создать случайный звук или мутацию, воспроизвести, экспортировать WAV, сохранить и загрузить параметры, вернуться к предыдущему варианту. Проверка файла параметров включена в сценарий загрузки, а сообщение об ошибке расширяет сценарии при некорректном файле или недоступном Web Audio API.

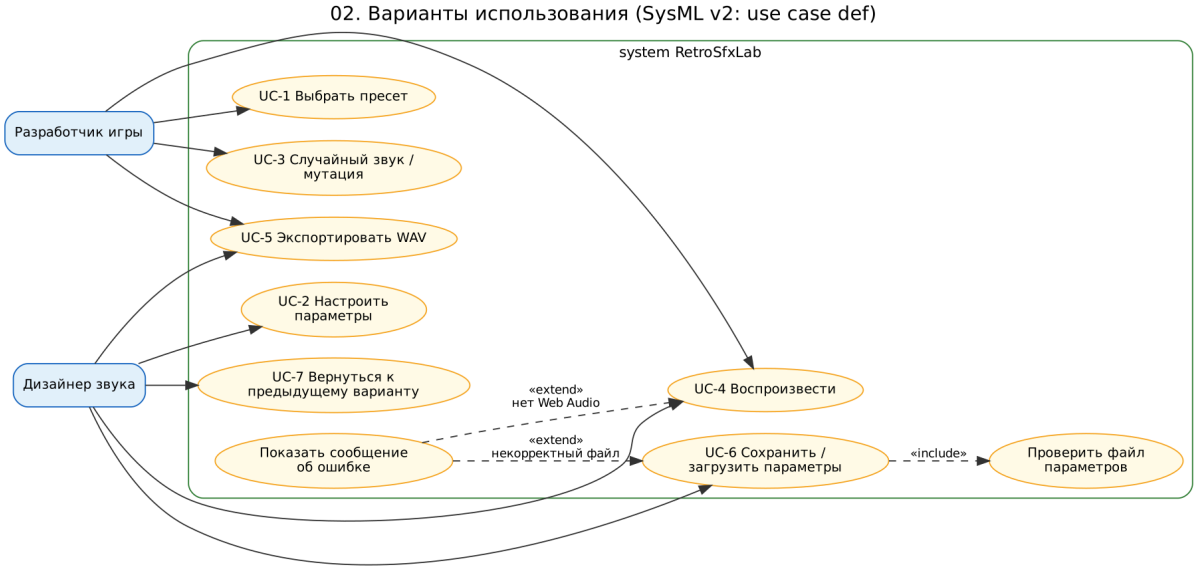


Рисунок 3.2 – Диаграмма вариантов использования Retro SFX Lab

3.4.3 Архитектура приложения

Архитектурная модель делит приложение на три части: клиентский интерфейс (Blazor WebAssembly), REST API (ASP.NET Core) и независимое ядро синтеза (.NET Class Library). Интерфейс содержит панель параметров, панель пресетов и график волны. Ядро включает параметры, синтезатор с осциллятором, огибающей и эффектами, пресеты, кодировщик WAV, сериализатор параметров и историю.

Ключевой принцип модели – ядро получает параметры и возвращает массив отсчётов, не завися от ASP.NET Core и веб-интерфейса. Поэтому ядро проверяется модульными тестами (xUnit) в изолированном окружении, а прослушанный и экспортированный звук формируются из одних и тех же отсчётов. Это также позволяет легко перенести ядро в Unity или Godot.

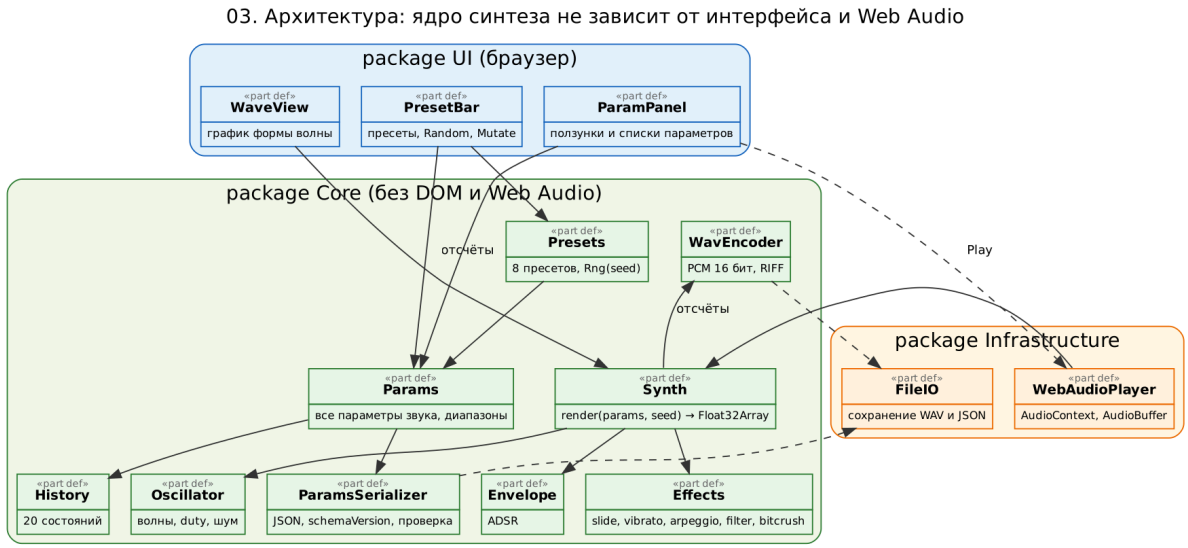


Рисунок 3.3 – Архитектура приложения Retro SFX Lab

3.4.4 Состояния редактора звука

Конечный автомат описывает работу редактора. Из состояния готовности изменение параметра переводит редактор в расчёт отсчётов, после обновления графика он возвращается в готовность. Воспроизведение и экспорт выполняются в отдельных состояниях; повторное нажатие «Play» перезапускает звук. Ошибка записи файла или загрузки некорректных параметров ведёт в состояние ошибки, из которого можно вернуться после закрытия сообщения.

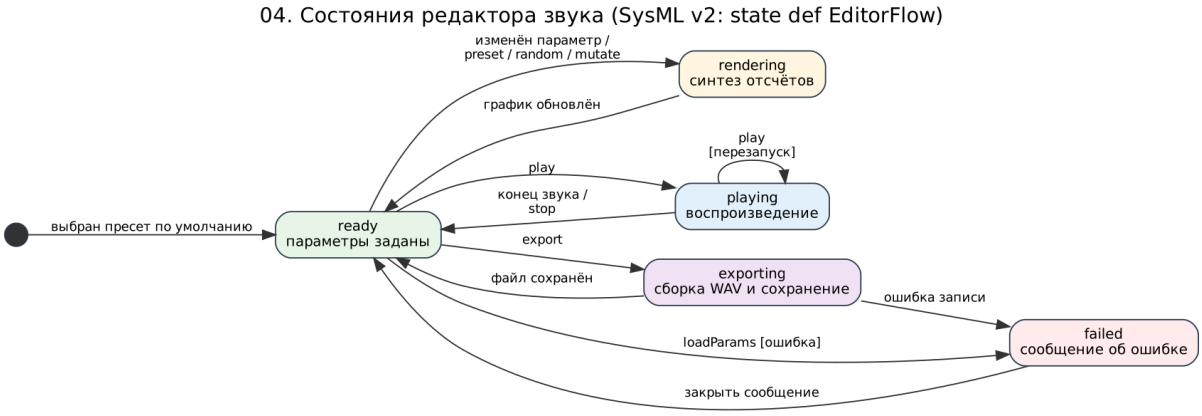
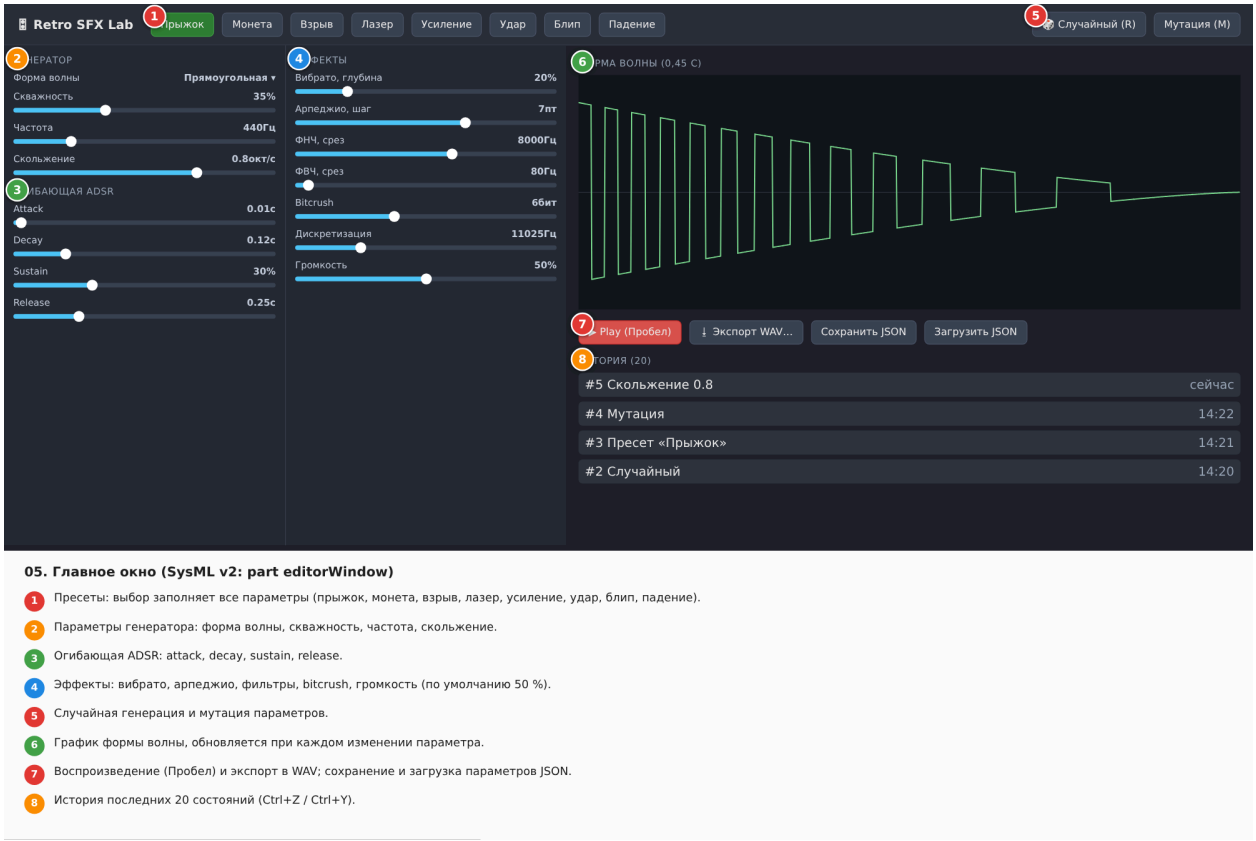


Рисунок 3.4 – Диаграмма состояний редактора звука

3.4.5 Макеты пользовательского интерфейса

Макеты показывают главное окно, диалог экспорта и панель пресетов. В главном окне находятся панель пресетов, параметры генератора и огибающей, эффекты, график формы волны, кнопки воспроизведения и экспорта, история.

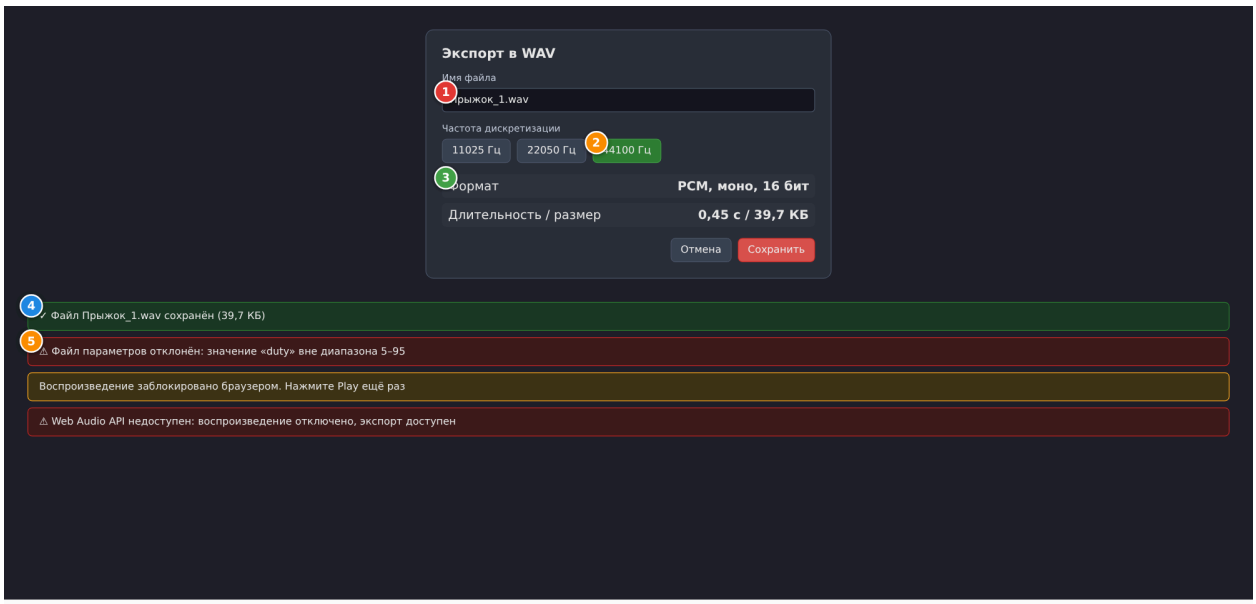
В диалоге экспорта выбираются имя файла и частота дискретизации; отдельно показаны сообщения об успехе, ошибках и блокировке воспроизведения.



05. Главное окно (SysML v2: part editorWindow)

1. Пресеты: выбор заполняет все параметры (прыжок, монета, взрыв, лазер, усиление, удар, блип, падение).
2. Параметры генератора: форма волны, скважность, частота, скольжение.
3. Огибающая ADSR: attack, decay, sustain, release.
4. Эффекты: вибрато, арпеджио, фильтры, bitcrush, громкость (по умолчанию 50 %).
5. Случайная генерация и мутация параметров.
6. График формы волны, обновляется при каждом изменении параметра.
7. Воспроизведение (Пробел) и экспорт в WAV; сохранение и загрузка параметров JSON.
8. История последних 20 состояний (Ctrl+Z / Ctrl+Y).

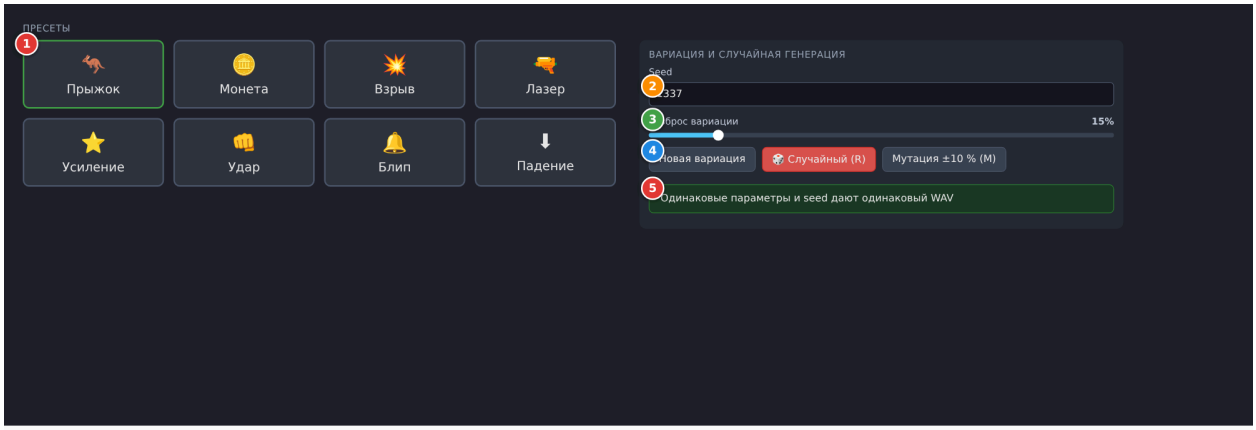
Рисунок 3.5 – Главное окно приложения



05. Диалог экспорта и сообщения

1. Имя файла формируется из названия пресета и номера вариации; недопустимые символы заменяются на «_».
2. Выбор частоты дискретизации: 11025, 22050 или 44100 Гц.
3. Формат фиксирован: PCM, моно, 16 бит; показаны длительность и размер.
4. Подтверждение успешного сохранения.
5. Сообщения об ошибках, блокировке автозапуска и отсутствии Web Audio API.

Рисунок 3.6 – Диалог экспорта и сообщения



05. Пресеты и случайная генерация

1. Сетка пресетов (8 шт.); выбранный пресет подсвечен.
2. Seed определяет воспроизводимые вариации пресета.
3. Разброс вариации: до ±15 % от параметров пресета.
4. Кнопки новой вариации, случайной генерации и мутации.
5. Пояснение: результат воспроизводим — один и тот же WAV при тех же параметрах.

Рисунок 3.7 – Пресеты и случайная генерация

3.4.6 Цепочка синтеза звука

Звук формируется последовательной обработкой в ядре на C#: частота модулируется скольжением, вибрато и арпеджио, осциллятор создаёт выбранную форму волны (для шума используется генератор с заданным seed), затем применяются огибающая ADSR, фильтры, bitcrush и ограничитель громкости.

Результат – массив отсчётов, из которого создаётся файл WAV для экспорта и формируются данные для визуализации формы волны на клиенте.

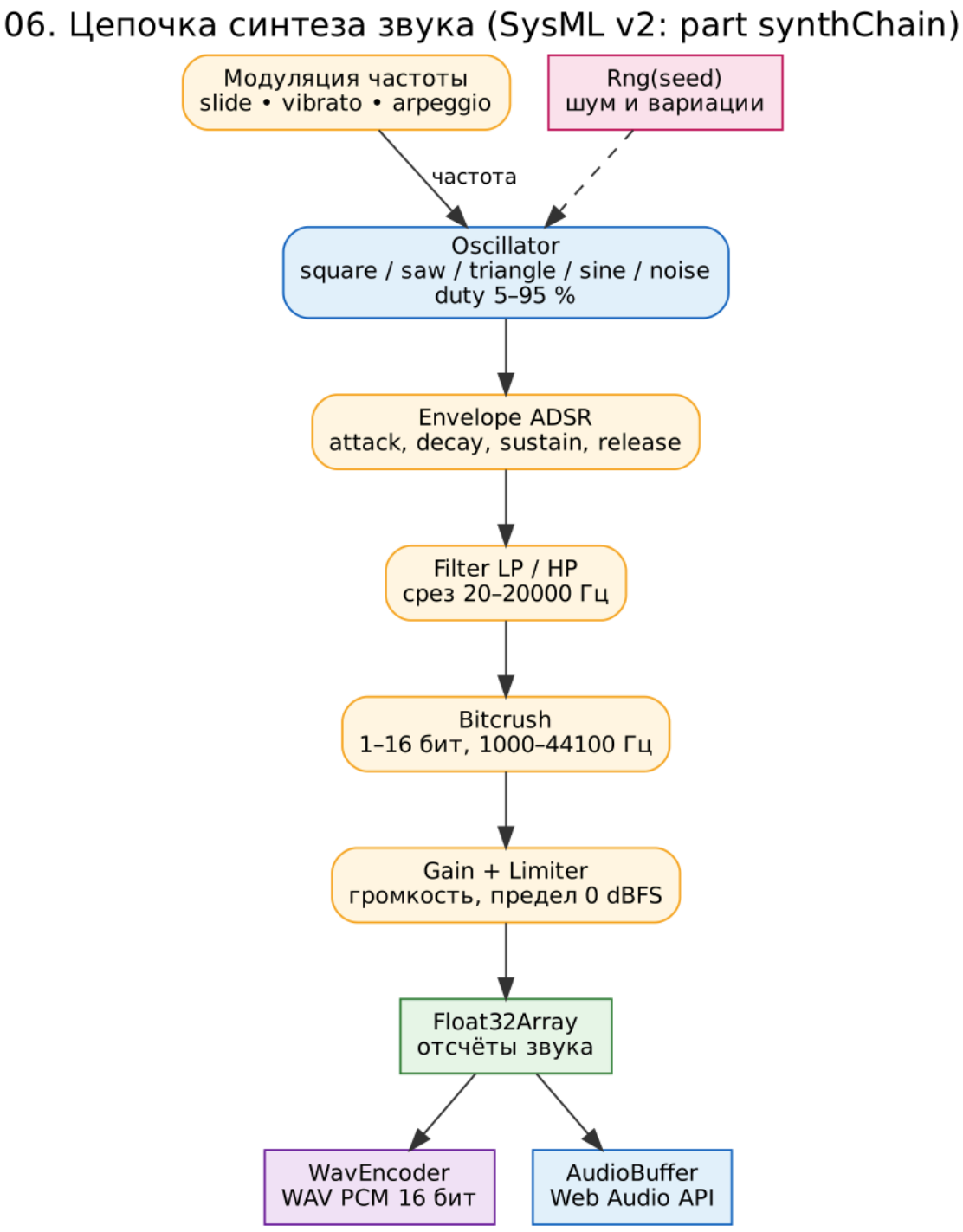


Рисунок 3.8 – Цепочка синтеза звука

3.5 Работа с репозиторием и артефактами

В комплект результатов входят спецификация требований `srs.md`, ис-

ходные модели SysML v2 и графические материалы в каталоге `document/diagrams/`. Спецификация оформлена по шаблону из репозитория лабораторной работы №2, а работа над документами велась в отдельной ветке.

Замечания и пожелания сначала Заказчика (напарника), а затем преподавателя оформляются в виде pull-request. Конструктивные замечания учитываются, а отклонённые сопровождаются обоснованием. Открытые вопросы заказчику выделены в приложении SRS, чтобы решения, которые ещё требуют подтверждения, были отделены от уже сформулированных требований.

4 РЕЗУЛЬТАТЫ РАБОТЫ

В результате выполнения работы подготовлена спецификация требований к веб-приложению «Retro SFX Lab» и комплект сопровождающих моделей SysML v2.

В комплект входят:

– SRS с 51 требованием, критериями приёмки и способами верификации;

– контекстная диаграмма и диаграмма вариантов использования;

– архитектурная диаграмма с разделением интерфейса, API и независимого ядра на C#;

– диаграмма состояний редактора звука;

– макеты пользовательского интерфейса и сообщений об ошибках;

– модель цепочки синтеза звука.

В документе отражены бизнес-цели, границы решения, ограничения реализации, требования к поведению и качеству. Для проверки предусмотрены тесты функционального поведения, производительности, безопасности и надёжности, а также демонстрации и инспекции.

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

5.1 Какие ресурсы ограничены и волнуют заказчика, заказывающего разработку ПО?

Заказчика обычно волнуют бюджет, сроки и доступные человеческие ресурсы, а также качество и объём функциональности итогового продукта. Эти факторы связаны: увеличение функциональности или качества может потребовать дополнительного времени и усилий. Поэтому требования и приоритеты необходимо согласовывать с заказчиком.

5.2 Хорошо ли, когда заказчик видит только один раз разработанное ПО, когда оно готово. Почему?

Это нежелательно. Если заказчик знакомится с результатом только в конце, несоответствие его ожиданиям может обнаружиться слишком поздно и потребовать существенных переделок. Макеты, модели и промежуточные демонстрации позволяют получать обратную связь на ранних этапах и уточнять требования до завершения реализации.

5.3 Стоит ли обращаться к заказчику на протяжении этапа сбора требований и разработки, если какие-либо из них не ясны. Что это даст?

Да. Уточнение требований помогает устранить неоднозначности, согласовать границы продукта и избежать решений, основанных на догадках разработчика. Это снижает риск реализации ненужной или неправильно понятой функции и упрощает приёмку результата.

5.4 Что такое требование к ПО?

Требование к ПО – это условие, характеристика или возможность, необходимая системе для удовлетворения потребности пользователя или достижения бизнес-цели. Требования могут описывать функции системы, ограничения и показатели качества. Хорошее требование должно быть однозначным, проверяемым и согласованным с остальными требованиями.

5.5 Как правило, обладает ли заказчик в начале разработки ПО ясным видением того, что должно получиться в итоге. Что сделать для того, чтобы уменьшить затраты заказчика?

Обычно в начале работы у заказчика есть общее представление о цели и основных возможностях продукта, но не все детали определены. Чтобы уменьшить затраты, следует обсуждать неоднозначные вопросы, фиксировать допущения, демонстрировать макеты и прототипы, а также согласовывать изменения до того, как они приведут к большой переделке уже реализованной системы.

5.6 Как правило, меняются ли требования во время разработки проекта? Как организовать работу с заказчиком так, чтобы свести затраты на переделывание ПО к минимуму?

Требования могут изменяться по мере уточнения потребностей, появления промежуточных результатов или изменения условий проекта. Для снижения затрат следует вести историю изменений, связывать требования с моделями и тестами, регулярно обсуждать результаты с заказчиком и проектировать систему так, чтобы изменения затрагивали ограниченное число компонентов. В данном проекте эту роль выполняют идентификаторы требований, матрица верификации и независимое от интерфейса ядро синтеза на C#.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были рассмотрены принципы выявления, формулирования и документирования требований к программному обеспечению. На примере генератора звуковых эффектов «Retro SFX Lab» определены назначение и границы продукта, выделены бизнес-цели, сформулированы требования с критериями приёмки и методами проверки.

Для описания решения подготовлены модели SysML v2: контекст системы, варианты использования, архитектура, состояния редактора, макеты интерфейса и цепочка синтеза. Отдельное внимание уделено воспроизводимости результата, корректности формата WAV, безопасной громкости и независимости ядра синтеза на C# от веб-интерфейса.

Полученный комплект требований и моделей создаёт основу для дальнейшего проектирования, реализации и проверки приложения. Открытые вопросы, отмеченные в SRS, требуют согласования перед окончательной фиксацией соответствующих критериев.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг кода

Файл SRS

```
# Спецификация требований к ПО

Для веб-приложения «Retro SFX Lab» — генератора звуковых эффектов в стиле 8-bit

Версия 0.1

Подготовил Мистюкович Даниил Васильевич, гр. 450501

БГУИР, кафедра электронных вычислительных машин

04.11.2026

## Оглавление

* [1. Введение](#1-введение)
  * [1.1 Назначение документа](#11-назначение-документа)
  * [1.2 Область применения продукта](#12-область-применения-продукта)
  * [1.3 Определения, акронимы и сокращения](#13-определения-акронимы-и-сокращения)
  * [1.4 Ссылки](#14-ссылки)
  * [1.5 Обзор документа](#15-обзор-документа)
* [2. Обзор продукта](#2-обзор-продукта)
  * [2.1 Перспектива продукта](#21-перспектива-продукта)
  * [2.2 Функции продукта](#22-функции-продукта)
  * [2.3 Ограничения продукта](#23-ограничения-продукта)
  * [2.4 Характеристики пользователей](#24-характеристики-пользователей)
  * [2.5 Допущения и зависимости](#25-допущения-и-зависимости)
  * [2.6 Распределение требований](#26-распределение-требований)
* [3. Требования](#3-требования)
  * [3.1 Внешние интерфейсы](#31-внешние-интерфейсы)
    * [3.1.1 Пользовательские интерфейсы](#311-пользовательские-интерфейсы)
    * [3.1.2 Аппаратные интерфейсы](#312-аппаратные-интерфейсы)
    * [3.1.3 Программные интерфейсы](#313-программные-интерфейсы)
  * [3.2 Функциональные требования](#32-функциональные-требования)
  * [3.3 Качество обслуживания](#33-качество-обслуживания)
    * [3.3.1 Производительность](#331-производительность)
    * [3.3.2 Безопасность](#332-безопасность)
    * [3.3.3 Надёжность](#333-надёжность)
```

    * [3.3.4 Доступность](#334-доступность)
    * [3.3.5 Наблюдаемость](#335-наблюдаемость)
  * [3.4 Соответствие стандартам](#34-соответствие-стандартам)
  * [3.5 Проектирование и реализация](#35-проектирование-и-реализация)
    * [3.5.1 Установка](#351-установка)
    * [3.5.2 Сборка и поставка](#352-сборка-и-поставка)
    * [3.5.3 Распределение](#353-распределение)
    * [3.5.4 Сопровождаемость](#354-сопровождаемость)
    * [3.5.5 Повторное использование](#355-повторное-использование)
    * [3.5.6 Переносимость](#356-переносимость)
    * [3.5.7 Стоимость](#357-стоимость)
    * [3.5.8 Сроки](#358-сроки)
    * [3.5.9 Доказательство концепции](#359-доказательство-концепции)
    * [3.5.10 Управление изменениями](#3510-управление-изменениями)
  * [3.6 ИИ/МО](#36-иимо)
* [4. Верификация](#4-верификация)
* [5. Приложения](#5-приложения)

История изменений

Name	Date	Reason For Changes	Version
Д.В. Мистюкович	04.11.2026	Первая редакция по бизнес-требованиям Заказчика	0.1

1. Введение

Документ описывает требования к веб-приложению «Retro SFX Lab» — генератору ретро-звуков (прыжки, монеты, взрывы, лазеры) для инди-игр. Звук синтезируется из параметров (форма волны, огибающая, скольжение частоты, фильтры) и сохраняется в файл WAV. Он написан для Заказчика, разработчика, тестировщика и преподавателя. Бизнес-цели и границы продукта приведены в разделе 1.2, ограничения и пользователи в разделе 2, проверяемые требования в разделе 3.

1.1 Назначение документа

SRS определяет, **что** модуль должен делать, а не **как** он будет реализован. Заказчик использует его, чтобы подтвердить, что требования отражают его потребности. Разработчик берёт из него требования к проектированию и реализации, тестировщик строит по нему приёмочные тесты (раздел 4). Связанные артефакты: исходный код проекта, `README.md` и графические модели в каталоге `document/diagrams/`.

1.2 Область применения продукта

Продукт: «Retro SFX Lab», версия 0.1. Состоит из ядра синтеза на C# (.NET 10), REST API на ASP.NET Core и пользовательского интерфейса на Blazor WebAssembly, работающего в браузере.

Назначение. Разработчик игры выбирает готовый пресет (прыжок, монета, взрыв и т. д.) или создаёт звук с нуля, настраивает параметры ползунками, сразу слышит результат и экспортирует его в WAV для использования в игре.

Бизнес-цели (источник: Заказчик):

ID	Бизнес-цель	Измеримый результат
BO-1	Разработчик быстро получает звук для прототипа игры	Звук из пресета создаётся и экспортируется не более чем за 3 действия
BO-2	Звук можно тонко настроить без знаний цифровой обработки сигналов	Каждый параметр имеет понятное название, диапазон и мгновенно слышимый эффект
BO-3	Результат пригоден для игровых движков	Файл WAV (16 бит, PCM) открывается в Unity, Godot и аудиоредакторах. Ядро на C# можно подключить к Unity/Godot напрямую.
BO-4	Удачные звуки можно сохранить и воспроизвести повторно	Один и тот же набор параметров даёт одинаковый WAV-файл
BO-5	Набор звуков проекта получается разнообразным	Кнопки случайной генерации и мутации создают вариации одного звука

Входит в область: синтез звука по параметрам, пресеты, случайная генерация и мутация, воспроизведение, визуализация формы волны, история изменений, экспорт в WAV, сохранение и загрузка параметров в JSON.

Не входит в область: запись с микрофона, импорт и редактирование чужих аудиофайлов, многодорожечный секвенсор, музыкальная композиция, экспорт в MP3/OGG, облачное хранилище и учётные записи, мобильные приложения.

1.3 Определения, акронимы и сокращения

Term	Definition
8-bit / чиптюн	Стиль звука игровых приставок 1980-х с простыми формами волны
ADSR	Attack, Decay, Sustain, Release — огибающая громкости: нарастание, спад, удержание, затухание
Bitcrush	Эффект уменьшения разрядности отсчётов, создающий «грязный» цифровой звук
Duty cycle (скважность)	Доля периода прямоугольной волны, в

```
течение которой сигнал высокий |
| PCM | Pulse-Code Modulation — несжатое представление звука
отсчётами |
| SPA | Single Page Application — одностраничное веб-приложение |
| SRS | Software Requirements Specification — спецификация требований
к ПО |
| Seed | Начальное число генератора случайных чисел; определяет
воспроизводимость случайных вариаций |
| Slide | Плавное изменение частоты звука во времени |
| SysML v2 | Systems Modeling Language версии 2 — язык моделирования
для графической части |
| TBD | To Be Determined — значение ещё не согласовано с Заказчиком |
| UI | User Interface — пользовательский интерфейс |
| WAV | Контейнерный формат звукового файла (RIFF/WAVE) |
| Web Audio API | Браузерный интерфейс для воспроизведения и
обработки звука (на клиенте) |
| Арпеджио | Резкая смена высоты звука через равные интервалы времени
|
| Вибрато | Периодическое колебание частоты звука |
| Пресет | Готовый набор параметров, соответствующий типовому
игровому звуку |

### 1.4 Ссылки

| № | Источник | Тип | Расположение |
|---|----------|-----|--------------|
| 1 | Источники и виды требований, ЖЦРПО Ч1 (СЭО БГУИР) | Справочная
| `Источники_и_виды_требований.html` |
| 2 | Документация .NET 8 и ASP.NET Core | Нормативная | https://
learn.microsoft.com/dotnet |
| 3 | Microsoft/IBM. Multimedia Programming Interface — спецификация
RIFF/WAVE | Нормативная | Формат WAV (PCM) |
| 4 | WCAG 2.1 — Web Content Accessibility Guidelines | Нормативная |
https://www.w3.org/TR/WCAG21/ |
| 5 | OMG Systems Modeling Language (SysML) v2 | Справочная |
https://www.omg.org/sysml/sysmlv2/ |
| 6 | Графические модели SysML v2 (исходники `.sysml` и PNG) |
Нормативная | `document/diagrams/` |

### 1.5 Обзор документа

Раздел 2 даёт контекст: место модуля в проекте, функции, ограничения,
пользователи, допущения и распределение требований по итерациям.
Раздел 3 содержит требования с уникальными идентификаторами `REQ-
[AREA]-[NNN]`; ключевое слово «должен» означает обязательное
требование, «следует» означает рекомендацию. Раздел 4 связывает
каждое требование со способом проверки. Раздел 5 содержит открытые
вопросы к Заказчику и список графических моделей. Каждое изменение
```

документа фиксируется в истории изменений.

---

2. Обзор продукта

2.1 Перспектива продукта

Приложение состоит из трех частей: независимое ядро синтеза на C# (.NET Class Library), REST API на ASP.NET Core (выполняет синтез и отдаёт WAV-файл или JSON) и интерфейс на Blazor WebAssembly. Воспроизведение звука происходит на клиенте через Web Audio API (JS Interop), а экспорт WAV выполняется на сервере (C#) или клиенте. Использование C# позволяет использовать ядро синтеза напрямую в игровых движках (Unity, Godot). Владелец проекта — автор репозитория.

Рисунок 1 — контекст системы (SysML v2: `document/diagrams/01-context.sysml`).

2.2 Функции продукта

1. Синтез звука: формы волны, скважность, частота, огибающая ADSR, скольжение, вибрато, арпеджио.
2. Фильтры низких и высоких частот, эффекты bitcrush и снижения частоты дискретизации.
3. Пресеты, случайная генерация и мутация параметров.
4. Воспроизведение и визуализация формы волны.
5. История изменений параметров (Undo/Redo).
6. Экспорт в WAV, сохранение и загрузка параметров в JSON.

Варианты использования показаны на рисунке 2.

Рисунок 2 — варианты использования (SysML v2: `document/diagrams/02-use-cases.sysml`).

2.3 Ограничения продукта

ID	Ограничение	Тип
CON-1	Язык разработки ядра и бэкенда — C# (.NET 8). Интерфейс — Blazor WebAssembly.	Внешнее, обязательное (Заказчик)
CON-2	Серверной БД нет; параметры и звуки не покидают компьютер пользователя (stateless API).	Внешнее (архитектурное)
CON-3	Поддерживаются актуальные версии Chrome, Firefox и Edge.

Внешнее |
CON-4	Длительность звука не более 5 секунд; моно, 44100 Гц по умолчанию.	Внешнее
CON-5	Внешние сервисы и платные библиотеки не используются.	Организационное
CON-6	Приложение формирует только синтезированные звуки и не обрабатывает чужие аудиофайлы.	Внешнее

2.4 Характеристики пользователей

Класс	Описание	Особенности
Разработчик игры	Создаёт звуки для своего проекта	Не обязательно знает синтез звука; ожидает пресеты и мгновенный отклик
Дизайнер звука	Тонко настраивает звук и сохраняет наборы параметров	Понимает ADSR и фильтры; ожидает точные значения и воспроизводимость
Разработчик приложения	Дорабатывает и сопровождает приложение	Знает C#; ожидает независимое ядро и тесты

2.5 Допущения и зависимости

ID	Допущение / зависимость	Влияние, если неверно
AS-1	Пользователь работает в браузере с поддержкой Web Audio API.	Воспроизведение не работает, экспорт доступен (см. REQ-INT-003).
AS-2	Воспроизведение начинается после первого действия пользователя.	Браузер блокирует автозапуск (см. REQ-REL-002).
AS-3	Размер звука ограничен 5 секундами.	Звук помещается в оперативную память.

2.6 Распределение требований

Трассируемость требований Заказчика:

№	Требование Заказчика	Требования SRS
C-1	Приложение запускается в браузере без ошибок	REQ-INT-001, REQ-INT-002, REQ-INT-003, REQ-REL-001
C-2	Синтез звука из параметров с огибающей и эффектами	REQ-FUNC-001 … REQ-FUNC-010, REQ-FUNC-023, REQ-PERF-001
C-3	Пресеты, случайная генерация и мутация	REQ-FUNC-011 … REQ-FUNC-014
C-4	Воспроизведение и визуализация	REQ-FUNC-015, REQ-FUNC-016, REQ-PERF-002, REQ-REL-002
C-5	Экспорт в WAV и сохранение параметров	REQ-FUNC-017 … REQ-FUNC-020, REQ-PERF-003, REQ-COMP-001
C-6	Воспроизводимость и защита от неверных данных	REQ-FUNC-021,

REQ-FUNC-022, REQ-SEC-*, REQ-REL-001, REQ-REL-003 |

Распределение по итерациям:

Итерация	Состав	Требования
1 (доказательство концепции)	Прототип ядра на C#: прямоугольная волна, ADSR и экспорт WAV	REQ-POC-001
2 (ядро)	Синтезатор, эффекты, пресеты, формат параметров	REQ-FUNC-001 … REQ-FUNC-014, REQ-FUNC-020 … REQ-FUNC-023, REQ-SEC-*
3 (интерфейс)	Панель параметров, воспроизведение, визуализация, история	REQ-INT-*, REQ-FUNC-015 … REQ-FUNC-019
4 (качество)	Производительность, наблюдаемость, тесты, документация	REQ-PERF-*, REQ-OBS-*, REQ-MAINT-*, REQ-COMP-*

Отложено: экспорт в OGG/MP3, стерео и панорама, пакетный экспорт набора звуков в ZIP (см. OQ-2).

---

3. Требования

Каждое требование содержит поля: **ID**, **Statement**, **Rationale**, **Acceptance Criteria**, **Verification Method** (Test, Analysis, Inspection, Demonstration). Источник по умолчанию — Заказчик; ссылки вида C-N указывают на таблицу в разделе 2.6.

3.1 Внешние интерфейсы

3.1.1 Пользовательские интерфейсы

Макеты: `document/diagrams/05-ui-main.png`, `document/diagrams/05-ui-export.png`, `document/diagrams/05-ui-presets.png`.

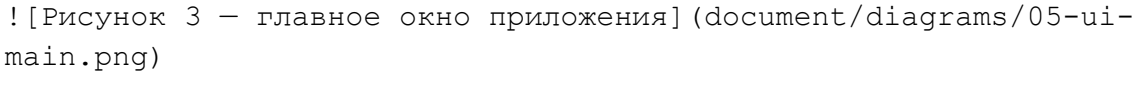


Рисунок 3 — главное окно приложения.

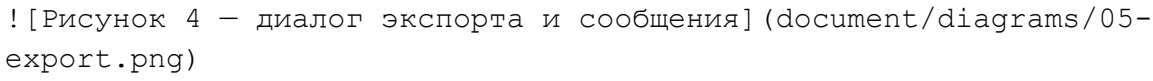


Рисунок 4 — диалог экспорта и сообщения.

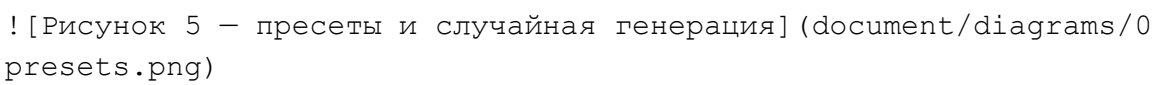


Рисунок 5 — пресеты и случайная генерация.

REQ-INT-001 — Поддерживаемые браузеры
- **Statement:** Приложение должно запускаться и работать в актуальных версиях Chrome, Firefox и Edge.
- **Rationale:** Заказчики используют разные браузеры.
- **Acceptance Criteria:** Сценарии C-1…C-6 выполняются в трёх браузерах без ошибок в консоли.
- **Verification Method:** Demonstration

REQ-INT-002 — Автономная работа
- **Statement:** Приложение должно работать без подключения к глобальной сети после загрузки статических файлов и запуска локального контейнера.
- **Rationale:** Работа без интернета.
- **Acceptance Criteria:** При отключённой сети доступны все функции раздела 3.2.
- **Verification Method:** Test

REQ-INT-003 — Отсутствие Web Audio API
- **Statement:** Если браузер не поддерживает Web Audio API, приложение должно сообщить об этом и сохранить доступными экспорт WAV и сохранение параметров.
- **Rationale:** Экспорт (выполняется на C# бэкенде) не зависит от воспроизведения.
- **Acceptance Criteria:** При отключённой поддержке воспроизведения кнопка Play неактивна, экспорт работает.
- **Verification Method:** Test

REQ-INT-004 — Управление с клавиатуры
- **Statement:** Приложение должно поддерживать клавиши: Пробел — воспроизвести, R — случайный звук, M — мутация, Ctrl+Z / Ctrl+Y — история, Ctrl+E — экспорт.
- **Rationale:** Быстрый подбор звука.
- **Acceptance Criteria:** Каждая клавиша вызывает действие; в полях ввода клавиши не срабатывают.
- **Verification Method:** Test

REQ-INT-005 — Размер экрана
- **Statement:** Интерфейс должен корректно отображаться при ширине окна от 1024 px.
- **Rationale:** Работа на ноутбуках и мониторах.
- **Acceptance Criteria:** Все органы управления доступны без горизонтальной прокрутки.
- **Verification Method:** Demonstration
- **More Information:** `document/diagrams/05-ui-main.png`.

3.1.2 Аппаратные интерфейсы

Аппаратных интерфейсов нет. Приложение работает на оборудовании пользователя.

3.1.3 Программные интерфейсы

REQ-INT-006 — REST-интерфейс экспорта WAV
- **Statement:** ASP.NET Core бэкенд должен предоставлять `POST /api/v1/sfx/export`, принимающий JSON с параметрами синтеза и возвращающий `audio/wav` с заголовком `Content-Disposition: attachment`.
- **Rationale:** Единый способ генерации и скачивания WAV.
- **Acceptance Criteria:** При корректных параметрах ответ имеет статус 200, `Content-Type: audio/wav`, тело начинается с `RIFF`. Для некорректных параметров — статус 400.
- **Verification Method:** Test

REQ-INT-007 — REST-интерфейс синтеза (Preview)
- **Statement:** Бэкенд должен предоставлять `POST /api/v1/sfx/preview`, возвращающий массив отсчётов (base64) для отрисовки графика формы волны на клиенте.
- **Rationale:** Тяжёлые расчёты выполняются на C#, клиент только рисует.
- **Acceptance Criteria:** Ответ содержит корректный массив чисел с плавающей точкой от -1.0 до 1.0.
- **Verification Method:** Test

3.2 Функциональные требования

Цепочка синтеза звука (от параметров до готового массива отсчётов) проиллюстрирована на Рисунке 6.

Рисунок 6 — цепочка синтеза звука (SysML v2: `document/diagrams/04-editor-flow.sysml`).

REQ-FUNC-001 — Формы волны
- **Statement:** Система должна синтезировать звук с формой волны: прямоугольная, пилообразная, треугольная, синусоидальная, шум.
- **Rationale:** Базовые тембры чиптюна (C-2).
- **Acceptance Criteria:** Для каждой формы волны график и спектр соответствуют ожидаемой форме.
- **Verification Method:** Test

REQ-FUNC-002 — Скважность
- **Statement:** Система должна позволять задать скважность прямоугольной волны от 5 до 95 %.
- **Rationale:** Изменение тембра «ширины» импульса.
- **Acceptance Criteria:** При скважности 50 % доли высокого и

низкого уровней равны; другие значения меняют долю.
- **Verification Method:** Test

REQ-FUNC-003 — Базовая частота
- **Statement:** Система должна позволять задать начальную частоту от 20 до 8000 Гц.
- **Rationale:** Высота звука.
- **Acceptance Criteria:** Измеренная частота первого периода равна заданной с погрешностью не более 1 %.
- **Verification Method:** Test

REQ-FUNC-004 — Огибающая ADSR
- **Statement:** Система должна применять огибающую громкости ADSR: attack, decay (по 0–2 с), уровень sustain (0–100 %), время удержания (0–2 с) и release (0–2 с); общая длительность не должна превышать 5 с.
- **Rationale:** Форма звука «удар», «писк», «взрыв».
- **Acceptance Criteria:** Амплитуда по времени соответствует заданной огибающей; сумма параметров > 5 с ограничивается.
- **Verification Method:** Test

REQ-FUNC-005 — Скольжение частоты (Slide)
- **Statement:** Система должна позволять задать скольжение частоты (−1…+1 октавы в секунду) и ускорение скольжения.
- **Rationale:** Прыжки, лазеры, падения.
- **Acceptance Criteria:** При положительном скольжении частота растёт, при отрицательном падает; значение 0 даёт постоянную частоту.
- **Verification Method:** Test

REQ-FUNC-006 — Вибрато
- **Statement:** Система должна позволять задать глубину (0–100 %) и частоту (0,1–20 Гц) вибрато.
- **Rationale:** Эффект «дрожащего» звука.
- **Acceptance Criteria:** При включении вибрато частота периодически отклоняется от базовой; при нулевой глубине вибрато нет.
- **Verification Method:** Test

REQ-FUNC-007 — Арпеджио
- **Statement:** Система должна позволять задать смещение частоты (−24…+24 полутона) и скорость смены.
- **Rationale:** Эффект монеты и бонуса.
- **Acceptance Criteria:** В момент смены частота изменяется скачком на заданное число полутонов.
- **Verification Method:** Test

REQ-FUNC-008 — Фильтры
- **Statement:** Система должна применять фильтры низких и высоких частот с частотой среза 20–20000 Гц.

- **Rationale:** Смягчение и «осветление» звука.
- **Acceptance Criteria:** При срезе 500 Гц ФНЧ ослабляет гармоники выше 500 Гц не менее чем на 12 дБ.
- **Verification Method:** Test

REQ-FUNC-009 — Bitcrush и децимация
- **Statement:** Система должна позволять снижать разрядность (1–16 бит) и частоту дискретизации эффекта (1000–44100 Гц).
- **Rationale:** Характерный «цифровой» звук.
- **Acceptance Criteria:** При 4 битах отсчёты принимают не более 16 уровней; при низкой частоте видны «ступеньки».
- **Verification Method:** Test

REQ-FUNC-010 — Громкость
- **Statement:** Система должна позволять задать общую громкость 0–100 % (по умолчанию 50 %).
- **Rationale:** Защита слуха и единый уровень.
- **Acceptance Criteria:** Пиковое значение отсчётов пропорционально выбранной громкости.
- **Verification Method:** Test

REQ-FUNC-011 — Пресеты
- **Statement:** Система должна содержать не менее 8 пресетов: прыжок, монета, взрыв, лазер, усиление, удар, блип, падение; выбор пресета заполняет все параметры.
- **Rationale:** Быстрый старт (BO-1).
- **Acceptance Criteria:** После выбора каждого пресета параметры соответствуют его описанию, звук воспроизводится.
- **Verification Method:** Test

REQ-FUNC-012 — Вариации пресета
- **Statement:** Система должна создавать вариацию пресета по seed: параметры отклоняются от базовых не более чем на ±15 %.
- **Rationale:** Разнообразие звуков одного типа.
- **Acceptance Criteria:** Разные seed дают разные звуки, один seed — одинаковый.
- **Verification Method:** Test

REQ-FUNC-013 — Случайная генерация
- **Statement:** Система должна создавать случайный набор параметров в допустимых диапазонах одной кнопкой.
- **Rationale:** Поиск неожиданных звуков (BO-5).
- **Acceptance Criteria:** Для 1000 запусков ни один параметр не выходит за диапазон.
- **Verification Method:** Test

REQ-FUNC-014 — Мутация
- **Statement:** Система должна изменять каждый параметр текущего

звука на случайную величину не более ±10 % диапазона.
- **Rationale:** Тонкая вариация удачного звука.
- **Acceptance Criteria:** После мутации все параметры в допустимых диапазонах и отличаются от исходных.
- **Verification Method:** Test

REQ-FUNC-015 — Воспроизведение
- **Statement:** Система должна воспроизводить звук кнопкой «Play» и по клавише Пробел; повторное нажатие перезапускает звук.
- **Rationale:** Быстрая проверка результата.
- **Acceptance Criteria:** Звук начинается не позднее чем через 100 мс; два нажатия подряд не создают наложения.
- **Verification Method:** Test

REQ-FUNC-016 — Визуализация
- **Statement:** Система должна отображать форму волны всего звука на графике и обновлять её при изменении параметров.
- **Rationale:** Наглядность влияния параметров.
- **Acceptance Criteria:** После изменения любого параметра график обновляется.
- **Verification Method:** Test
- **More Information:** `document/diagrams/04-editor-flow.png`.

REQ-FUNC-017 — Экспорт WAV
- **Statement:** Система должна экспортировать звук в файл WAV (PCM, моно, 16 бит) с частотой 11025, 22050 или 44100 Гц на выбор.
- **Rationale:** Использование в игровых движках (BO-3).
- **Acceptance Criteria:** Файл открывается в стороннем аудиоредакторе, заголовок и длительность соответствуют параметрам.
- **Verification Method:** Test

REQ-FUNC-018 — Имя файла
- **Statement:** Система должна предлагать имя файла по названию пресета и номеру вариации; недопустимые символы заменяются подчёркиванием.
- **Rationale:** Удобство и совместимость с ОС.
- **Acceptance Criteria:** Для пресета «Прыжок/1» предлагается `Прыжок_1.wav`.
- **Verification Method:** Test

REQ-FUNC-019 — Сохранение параметров
- **Statement:** Система должна сохранять текущие параметры в файл JSON.
- **Rationale:** Повторное использование звуков (C-5).
- **Acceptance Criteria:** Файл создаётся по команде и содержит все параметры и версию схемы.
- **Verification Method:** Test

REQ-FUNC-020 — Загрузка параметров
- **Statement:** Система должна загружать параметры из JSON, проверяя структуру, версию и диапазоны значений; при ошибке текущие параметры не меняются.
- **Rationale:** Файлы могут быть повреждены.
- **Acceptance Criteria:** Корректный файл восстанавливает звук; для некорректного — выводится сообщение.
- **Verification Method:** Test

REQ-FUNC-021 — Воспроизводимость
- **Statement:** При одинаковых параметрах и seed система должна создавать побайтно идентичные файлы WAV.
- **Rationale:** Повторяемость результата (BO-4).
- **Acceptance Criteria:** Два экспорта подряд дают файлы с одинаковой контрольной суммой SHA-256.
- **Verification Method:** Test

REQ-FUNC-022 — История
- **Statement:** Система должна хранить до 20 последних состояний параметров и позволять возвращаться к ним (отмена и повтор).
- **Rationale:** Возврат к удачному варианту.
- **Acceptance Criteria:** Ctrl+Z возвращает предыдущее состояние, Ctrl+Y повторяет изменение.
- **Verification Method:** Test

REQ-FUNC-023 — Защита от перегрузки
- **Statement:** Система должна ограничивать пиковое значение звука до 0 dBFS без выхода за границы 16 бит.
- **Rationale:** Отсутствие искажений и щелчков при переполнении.
- **Acceptance Criteria:** В экспортированном файле нет значений, вышедших за диапазон; при максимальных параметрах нет переполнения.
- **Verification Method:** Test

3.3 Качество обслуживания

3.3.1 Производительность

REQ-PERF-001 — Скорость синтеза
- **Statement:** Синтез звука длительностью 1 секунда на C# должен занимать не более 100 мс.
- **Rationale:** Мгновенный отклик при подборе параметров.
- **Acceptance Criteria:** Время расчёта для 1 с звука не превышает 100 мс на ноутбуке среднего уровня.
- **Verification Method:** Test

REQ-PERF-002 — Отклик интерфейса
- **Statement:** После изменения параметра график должен обновляться не позднее чем через 100 мс.

- **Rationale:** Плавная работа с ползунками.
- **Acceptance Criteria:** Задержка между изменением ползунка и обновлением графика не превышает 100 мс.
- **Verification Method:** Test

REQ-PERF-003 — Скорость экспорта
- **Statement:** Экспорт звука длительностью 5 секунд в WAV должен выполняться не более чем за 500 мс.
- **Rationale:** Быстрое сохранение.
- **Acceptance Criteria:** Время от нажатия кнопки до появления файла не превышает 500 мс.
- **Verification Method:** Test

3.3.2 Безопасность

REQ-SEC-001 — Отсутствие внешних запросов
- **Statement:** Приложение не должно отправлять данные пользователя во внешние сети.
- **Rationale:** Параметры и звуки — собственность автора.
- **Acceptance Criteria:** Политика CSP запрещает внешние соединения; в панели сети нет запросов при работе.
- **Verification Method:** Inspection

REQ-SEC-002 — Проверка файла параметров
- **Statement:** Система должна проверять загружаемый JSON: размер не более 64 КБ, типы и диапазоны значений, отсутствие неизвестных полей.
- **Rationale:** Защита от некорректных файлов.
- **Acceptance Criteria:** Файлы 100 КБ, со строкой вместо числа и с лишними полями отклоняются с сообщением.
- **Verification Method:** Test

3.3.3 Надёжность

REQ-REL-001 — Некорректные данные
- **Statement:** Некорректные параметры и файлы не должны приводить к аварийному завершению приложения.
- **Rationale:** Файлы приходят от разных источников.
- **Acceptance Criteria:** Для 10 негативных файлов выводится сообщение, приложение работает.
- **Verification Method:** Test

REQ-REL-002 — Автозапуск звука
- **Statement:** Если браузер блокирует воспроизведение до действия пользователя, система должна возобновить звук после первого клика без потери параметров.
- **Rationale:** Политика автозапуска браузеров.
- **Acceptance Criteria:** После блокировки первое нажатие «Play» воспроизводит звук.

- **Verification Method:** Test

REQ-REL-003 — Безопасная громкость
- **Statement:** Громкость при запуске приложения и при загрузке пресета не должна превышать 50 %.
- **Rationale:** Защита слуха и динамиков.
- **Acceptance Criteria:** После загрузки приложения и пресета громкость не выше 50 %.
- **Verification Method:** Test

3.3.4 Доступность

Отдельных требований нет: приложение работает локально в браузере пользователя и не зависит от доступности серверов.

3.3.5 Наблюдаемость

REQ-OBS-001 — Журнал ошибок
- **Statement:** Бэкенд на ASP.NET Core должен записывать в консоль (ILogger) ошибки и предупреждения с кодом, уровнем и описанием.
- **Rationale:** Диагностика проблем.
- **Acceptance Criteria:** Для ошибок REQ-FUNC-020 и REQ-REL-002 в консоли есть записи с кодом.
- **Verification Method:** Inspection

3.4 Соответствие стандартам

REQ-COMP-001 — Формат WAV
- **Statement:** Файл WAV должен соответствовать спецификации RIFF/WAVE (PCM, формат 1) с корректными полями размеров.
- **Rationale:** Совместимость с движками.
- **Acceptance Criteria:** Файл успешно проверяется утилитой анализа и читается в трёх сторонних программах.
- **Verification Method:** Test

REQ-COMP-002 — Доступность интерфейса
- **Statement:** Контраст текста и фона элементов управления должен быть не менее 4.5:1 (WCAG 2.1, критерий 1.4.3), ползунки доступны с клавиатуры.
- **Rationale:** Читаемость и доступность.
- **Acceptance Criteria:** Проверка инструментом аудита не выявляет нарушений; ползунки управляются стрелками.
- **Verification Method:** Inspection

REQ-COMP-003 — Формат параметров
- **Statement:** Файл параметров должен быть корректным JSON (RFC 8259, UTF-8) с полем `schemaVersion`.
- **Rationale:** Совместимость и расширяемость.

- **Acceptance Criteria:** Файл разбирается стандартным парсером (System.Text.Json); поле версии присутствует.
- **Verification Method:** Test

3.5 Проектирование и реализация

3.5.1 Установка

REQ-INT-008 — Запуск проекта
- **Statement:** Проект должен запускаться командой `docker compose up --build` без ручных шагов.
- **Rationale:** Упрощение развертывания.
- **Acceptance Criteria:** После запуска API отвечает на `/health` статусом `Healthy`, интерфейс доступен.
- **Verification Method:** Demonstration

3.5.2 Сборка и поставка

Сборка выполняется командой `dotnet publish`; результат — готовые к запуску бинарные файлы и статические файлы Blazor.

3.5.3 Распределение

Исходный код хранится в Git-репозитории; изменения вносятся через pull-request.

3.5.4 Сопровождаемость

REQ-MAINT-001 — Тестовое покрытие
- **Statement:** Модульные тесты (xUnit) должны покрывать не менее 80 % строк ядра (синтезатор, эффекты, WAV, формат параметров).
- **Rationale:** Безопасные изменения.
- **Acceptance Criteria:** Отчёт покрытия (coverlet) показывает не менее 80 %.
- **Verification Method:** Analysis

REQ-MAINT-002 — Независимость ядра
- **Statement:** Ядро синтеза (Class Library) не должно зависеть от ASP.NET Core и веб-интерфейса; оно получает параметры и возвращает массив отсчётов.
- **Rationale:** Тесты без запуска сервера, возможность портирования в Unity.
- **Acceptance Criteria:** Каталог `Core/` не содержит ссылок на `Microsoft.AspNetCore.*`; тесты запускаются в изолированном окружении.
- **Verification Method:** Inspection

3.5.5 Повторное использование

Архитектура изоляции показана на рисунке 7.

Рисунок 7 — изоляция модуля (SysML v2: `document/diagrams/03-architecture.sysml`).

REQ-REUSE-001 — Ядро как NuGet-пакет
- **Statement:** Ядро синтеза должно быть оформлено как независимый NuGet-пакет (или dll), пригодный для использования в игре на C# (Unity, Godot).
- **Rationale:** Повторное использование в проектах Заказчика (BO-3).
- **Acceptance Criteria:** Пример использования ядра вне веб-интерфейса (консольное приложение) генерирует WAV.
- **Verification Method:** Demonstration

REQ-REUSE-002 — Версионирование параметров
- **Statement:** Формат параметров должен содержать версию схемы; загрузка файлов старых версий должна поддерживаться миграцией.
- **Rationale:** Совместимость сохранённых звуков.
- **Acceptance Criteria:** Файл версии 1 корректно загружается в версии 2 после миграции.
- **Verification Method:** Test

REQ-REUSE-003 — Подключение новой системы (пресета)
- **Statement:** Подключение нового пресета должно требовать только добавления JSON-файла или одного класса, реализующего интерфейс модуля.
- **Rationale:** BO-1.
- **Acceptance Criteria:** Новый пресет добавлен без изменения кода ядра и отображается в интерфейсе.
- **Verification Method:** Test

REQ-REUSE-004 — Подписи вне кода
- **Statement:** Названия параметров и подписи интерфейса должны храниться во внешних ресурсах, а не в коде отрисовки.
- **Rationale:** Подготовка к локализации.
- **Acceptance Criteria:** Изменение подписи выполняется правкой ресурсного файла без изменения кода.
- **Verification Method:** Inspection

3.5.6 Переносимость

REQ-PORT-001 — Операционные системы
- **Statement:** Бэкенд на .NET 8 должен работать на Windows, macOS и Linux (через Docker).
- **Rationale:** Разные компьютеры разработчиков.

- **Acceptance Criteria:** Сценарии C-1…C-6 выполнены на трёх системах.
- **Verification Method:** Demonstration

3.5.7 Стоимость

Проект учебный: платные сервисы и лицензии не используются, все зависимости имеют открытые лицензии.

3.5.8 Сроки

Срок определяется графиком дисциплины; итерации приведены в разделе 2.6.

3.5.9 Доказательство концепции

REQ-POC-001 — Прототип
- **Statement:** На первой итерации должен быть создан прототип на C#: прямоугольная волна, огибающая ADSR, воспроизведение и экспорт в WAV.
- **Rationale:** Проверка основных технических рисков (синтез, формат WAV).
- **Acceptance Criteria:** Прототип демонстрируется преподавателю: созданный WAV открывается в аудиоредакторе и звучит так же.
- **Verification Method:** Demonstration

3.5.10 Управление изменениями

Изменения требований оформляются новой версией SRS и записью в таблице «История изменений». Замечания Заказчика и преподавателя вносятся через pull-request; отклонённые замечания сопровождаются обоснованием.

3.6 ИИ/МО

Методы искусственного интеллекта и машинного обучения в продукте не используются.

---

4. Верификация

Тестовые данные: набор валидных и невалидных JSON-файлов параметров, эталонные WAV-файлы. Окружение: `docker compose` проекта, Chrome/Edge и Firefox последних версий.

| Requirement ID | Verification Method | Test/Artifact Link | Status | Evidence |

```
|----------------|--------------------|--------------------|--------|------
| REQ-INT-001 | Demonstration | tests/manual/browsers.md | Planned |
|
| REQ-INT-002 | Test | tests/int/OfflineTest | Planned | |
| REQ-INT-003 | Test | tests/int/NoAudioApiTest | Planned | |
| REQ-INT-004 | Test | tests/ui/HotkeysTest | Planned | |
| REQ-INT-005 | Demonstration | tests/manual/responsive.md | Planned
| |
| REQ-INT-006 | Test | tests/api/ExportEndpointTest | Planned | |
| REQ-INT-007 | Test | tests/api/PreviewEndpointTest | Planned | |
| REQ-INT-008 | Demonstration | tests/integration/startup.md |
Planned | |
| REQ-FUNC-001 | Test | tests/core/WaveformTest | Planned | |
| REQ-FUNC-002 | Test | tests/core/DutyCycleTest | Planned | |
| REQ-FUNC-003 | Test | tests/core/FrequencyTest | Planned | |
| REQ-FUNC-004 | Test | tests/core/AdsrTest | Planned | |
| REQ-FUNC-005 | Test | tests/core/SlideTest | Planned | |
| REQ-FUNC-006 | Test | tests/core/VibratoTest | Planned | |
| REQ-FUNC-007 | Test | tests/core/ArpeggioTest | Planned | |
| REQ-FUNC-008 | Test | tests/core/FilterTest | Planned | |
| REQ-FUNC-009 | Test | tests/core/BitcrushTest | Planned | |
| REQ-FUNC-010 | Test | tests/core/VolumeTest | Planned | |
| REQ-FUNC-011 | Test | tests/core/PresetsTest | Planned | |
| REQ-FUNC-012 | Test | tests/core/SeedVariationTest | Planned | |
| REQ-FUNC-013 | Test | tests/core/RandomGenTest | Planned | |
| REQ-FUNC-014 | Test | tests/core/MutationTest | Planned | |
| REQ-FUNC-015 | Test | tests/ui/PlaybackTest | Planned | |
| REQ-FUNC-016 | Test | tests/ui/VisualizationTest | Planned | |
| REQ-FUNC-017 | Test | tests/core/WavExportTest | Planned | |
| REQ-FUNC-018 | Test | tests/core/FileNameTest | Planned | |
| REQ-FUNC-019 | Test | tests/core/JsonSaveTest | Planned | |
| REQ-FUNC-020 | Test | tests/core/JsonLoadTest | Planned | |
| REQ-FUNC-021 | Test | tests/core/ReproducibilityTest | Planned | |
| REQ-FUNC-022 | Test | tests/ui/HistoryTest | Planned | |
| REQ-FUNC-023 | Test | tests/core/ClippingTest | Planned | |
| REQ-PERF-001 … 003 | Test | tests/perf/BenchmarkTests | Planned | |
| REQ-SEC-001 | Inspection | checklist/csp.md | Planned | |
| REQ-SEC-002 | Test | tests/core/JsonValidationTest | Planned | |
| REQ-REL-001 | Test | tests/core/FuzzTest | Planned | |
| REQ-REL-002 | Test | tests/ui/AutoplayTest | Planned | |
| REQ-REL-003 | Test | tests/ui/VolumeTest | Planned | |
| REQ-OBS-001 | Inspection | checklist/logging.md | Planned | |
| REQ-COMP-001 | Test | tests/core/WavSpecTest | Planned | |
| REQ-COMP-002 | Inspection | checklist/a11y.md | Planned | |
| REQ-COMP-003 | Test | tests/core/JsonSchemaTest | Planned | |
| REQ-MAINT-001 | Analysis | coverlet/report | Planned | |
| REQ-MAINT-002 | Inspection | tests/arch/IsolationTest | Planned | |
| REQ-REUSE-001 | Demonstration | samples/ConsoleApp | Planned | |
```

```
| REQ-REUSE-002 | Test | tests/core/MigrationTest | Planned | |
| REQ-REUSE-003 | Test | tests/core/NewPresetTest | Planned | |
| REQ-REUSE-004 | Inspection | checklist/labels.md | Planned | |
| REQ-PORT-001 | Demonstration | tests/manual/port.md | Planned | |
| REQ-POC-001 | Demonstration | poc/console-prototype | Planned | |

---

## 5. Приложения

### Приложение A. Открытые вопросы к Заказчику (TBD)

| ID | Вопрос | Предлагаемое решение по умолчанию | Затронутые
требования |
|----|--------|-----------------------------------|----------------------|
| OQ-1 | Нужно ли стерео и панорамирование? | Нет, только моно | REQ-
FUNC-017 |
| OQ-2 | Нужен ли пакетный экспорт набора звуков в ZIP? | Нет,
отложить | — |
| OQ-3 | Нужен ли экспорт в OGG/MP3? | Нет, только WAV | REQ-FUNC-017
|
| OQ-4 | Достаточно ли ограничения 5 с для звуков Заказчика? | Да;
для музыки приложение не предназначено | REQ-FUNC-004 |
| OQ-5 | Нужна ли библиотека пользовательских пресетов в браузере? |
Нет, только файлы JSON | REQ-FUNC-019, REQ-FUNC-020 |
| OQ-6 | Какие ещё пресеты нужны (меню, шаги, дверь)? | Добавить
после первой версии | REQ-FUNC-011 |

### Приложение B. Графические модели SysML v2

Все диаграммы описаны в текстовой нотации SysML v2 (файлы `.sysml`),
PNG-файлы подготовлены на основе этих моделей; макеты интерфейса
(рисунки 3–5) построены на основе UI-китов.

| Рисунок | Назначение | PNG | Исходная модель |
|---------|------------|-----|-----------------|
| 1 | Контекст системы | `document/diagrams/01-context.png` |
`document/diagrams/01-context.sysml` |
| 2 | Варианты использования | `document/diagrams/02-use-cases.png` |
`document/diagrams/02-use-cases.sysml` |
| 3 | Главное окно приложения | `document/diagrams/05-ui-main.png` |
`document/diagrams/05-ui-mockups.sysml` |
| 4 | Диалог экспорта и сообщения | `document/diagrams/05-ui-
export.png` | `document/diagrams/05-ui-mockups.sysml` |
| 5 | Пресеты и случайная генерация | `document/diagrams/05-ui-
presets.png` | `document/diagrams/05-ui-mockups.sysml` |
| 6 | Состояния процесса экспорта | `document/diagrams/04-editor-
flow.png` | `document/diagrams/04-editor-flow.sysml` |
```

| 7 | Архитектура и изоляция модуля | `document/diagrams/03-architecture.png` | `document/diagrams/03-architecture.sysml` |
| 8 | Цепочка синтеза звука | `document/diagrams/06-synth-chain.png` | `document/diagrams/06-synth-chain.sysml` |

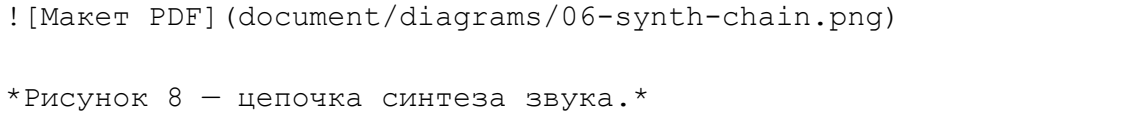


Рисунок 8 — цепочка синтеза звука.